

Organização e Arquitetura de Computadores

Jorge Augusto Salgado Salhani

no. USP: 8927418

Julho, 2026

1 Lista 5

1. Qual a principal razão de se ter um sistema de memória do computador organizado de maneira hierárquica?

[SOLUÇÃO]

Sistema de memória hierárquico permite a implementação de diferentes tipos de memória com vantagens e desvantagens combinadas. Memórias rápidas apresentam custo elevado e tem pouca capacidade de armazenamento. Memórias lentas tem baixo custo mas com alta capacidade. Combinando memórias de ambos tipos em um computador, temos um melhor balanço de armazenamento.

Também na implementação de memórias hierárquicas, é possível resgatar dados quanto à sua localização temporal ou espacial. Temporalmente, é possível considerar que dados recentemente acessados serão utilizados em breve, e espacialmente é possível considerar que dado fisicamente próximos a um dado recém lido também serão utilizados em breve.

2. Qual a diferença entre memórias voláteis e memórias não voláteis?

[SOLUÇÃO]

A volatilidade da memória diz respeito à característica de um dado ser mantido de forma perene ou não quando na ausência de energia. Memórias voláteis (RAM) existem enquanto os programas estão em execução. Já memórias não-voláteis (SSD, HDD) se mantém presentes mesmo na ausência de energia, com o computador desligado.

3. O que diferencia uma ROM de uma EPROM de uma E²PROM de uma FLASH?

[SOLUÇÃO]

ROM: Read Only Memory. Consiste em memória de apenas leitura, com dados gravados por padrão de fábrica,

sem possibilidade de alteração

EPROM: Erasable Programmable ROM. Consiste em memória de apenas leitura com capacidade de modificação via estímulo externo (aplicação de pulso magnético ou fonte UV). Não é modificável internamente ao computador

EEPROM: Electrically Erasable Programmable ROM. Similar à EPROM, mas permite reutilização de memória via elétrica, o que permite a remoção e escrita de dados sem remoção da componente do computador. Seu uso, no entanto, é limitado por escrita byte a byte.

FLASH: Análoga à E²PROM, mas com escrita de dados por setores, ao invés de bytes.

4. Qual a diferença entre uma RAM estática e uma RAM dinâmica?

[SOLUÇÃO]

Ambas memórias RAM são voláteis, portanto precisam de energia para manter os dados intactos. As DRAM (Dynamic RAM) utilizam capacitores, e precisam de carga contínua, o que torna a DRAM menor (e por isso mais DRAM por unidade de área pode ser implementado) mas mais lenta que em relação à SRAM, que por sua vez é implementada com flip-flops, com cerca de 6 transistores. Por não precisar de carga contínua, sua utilização é mais rápida, porém por ser necessário mais componentes, há menor densidade de memória.

5. Suponha que uma CPU tenha uma cache nível 1 e uma cache nível 2, com tempos de acesso de 5 e 10 ns, respectivamente. A memória principal tem tempo de acesso de 50 ns. e o cache nível 1 possui taxa de acertos de 20% e o cache nível 2 uma taxa de acertos de 60%, qual o tempo médio de acesso?

[SOLUÇÃO]

Considerando que o tempo médio de acesso é dado por

$$T = \sum_i x_i p'_i$$

onde x_i tempo de leitura, p'_i probabilidade de leitura correta (taxa de acerto), temos

$$T = x_{L1} p'_{L1} + x_{L2} p'_{L2} + x_{RAM} p'_{RAM}$$

Onde

$$p'_{L1} = p_{L1}$$

$$p'_{L2} = (1 - p_{L1}) p_{L2}$$

$$p'_{RAM} = (1 - p_{L1})(1 - p_{L2}) p_{RAM}$$

Visto que p'_{L2} é a probabilidade de errar cache 1 e acertar cache 2 e p'_{RAM} é a probabilidade de errar cache 1, cache 2 e acertar na RAM (com $p_{RAM} = 1$).

Assim

$$\begin{aligned} T &= [5 \times 0.2] + [10 \times (1 - 0.2) \times 0.6] + [50 \times (1 - 0.2)(1 - 0.6)] \\ &= 1 + 7.2 + 20.8 = 29ns \end{aligned}$$

6. Uma cache pode ser organizada de tal modo que se tenha:

- Mais palavras por bloco, mas poucos blocos
- Menos palavras por bloco, ms muitos blocos

Considere que nas duas opções a capacidade de cache (em bytes) é a mesma. Quais as vantagens e desvantagens de cada uma das abordagens? Dê um exemplo de sequência de acesso que tem uma taxa de acerto melhor em cada uma das opções.

[SOLUÇÃO]

Considerando uma capacidade fixa de memória, temos a relação $[\text{bytes}] = [\text{bytes}/\text{bloco}] \times [\text{nro_bloco}]$.

Caso tenhamos mais palavras por bloco (bytes/bloco), cada bloco conterá um número grande de palavras, portanto se a busca ocorrer por bytes fisicamente próximos, menor será o número de erros, com maior eficiência de uso de memória.

É desvantagem quando apenas poucas palavras de um carregamento são utilizadas.

No caso de leitura de um array, com blocos de 5 palavras, com até 2 blocos permitidos, dada a sequência de endereços $[0,1,2,3,4,5,6,7]$

- Acesso 0: Miss - Carrega em cache o bloco de $[0,1,2,3,4]$
- Acesso 1 a 4: Hit
- Acesso 5: Miss - Carrega em cache o bloco de $[5,6,7]$
- Acesso 6 a 7: Hit

Caso tenhamos poucas palavras por bloco, mas muitos blocos, o carregamento de novas palavras é mais rápido, visto que cada bloco trará poucos bytes e a substituição de cada bloco é menos frequente. Priorizamos assim a reutilização de palavras ao longo do tempo.

É desvantajoso caso percorremos arrays, mas vantajoso quando precisamos utilizar diversas vezes uma mesma variável.

Por exemplo, dada a sequência de endereços $[0,1,2,3,2,1,2,5,6,7]$, com blocos de 2 palavras e mantendo 10 palavras no total (portanto 5 blocos), temos

- Acesso 0: Miss - Carrega bloco [0,1]
- Acesso 1: Hit
- Acesso 2: Miss - Carrega bloco [2,3]
- Acesso 3,2,1,2: Hit
- Acesso 5: Miss - Carrega bloco [5,6]
- Acesso 6: Hit
- Acesso 7: Miss - Carrega bloco [7]

7. Para cada uma das caches definidas abaixo, faça a divisão do endereço nos campos necessários para os mapeamentos direto, associativo e associativo por conjunto ($k=4$). Considere que a memória principal é endereçada a Byte e que endereça 2^{32} bytes, portanto são necessários 32 bits para endereçar qualquer posição da memória. Considere também que uma *word* (palavra) tem 4 bytes.

- Cache com 16 words, 1 word por bloco (16 linhas)
- Cache com 32 words, 1 word por bloco (32 linhas)
- Cache com 64 words, 1 word por bloco (64 linhas)
- Cache com 128 words, 2 words por bloco (64 linhas)
- Cache com 128 words, 4 words por bloco (32 linhas)
- Cache com 1024 linhas, 8 words por bloco (8192 words)
- Cache com 256 linhas, 2 words por bloco (512 words)

[SOLUÇÃO]

O mapeamento consiste em uma forma de localizar uma palavra em uma sequência de posições de memória, separa em blocos, linhas, palavras, bytes.

No mapeamento direto, cada linha da cache vincula-se a um endereço físico na RAM

$$\text{LinhaCache} = \text{NumeroDoBloco} \bmod \text{TotalLinhasCache}$$

No caso associativo, o mapeamento da memória principal pode ser colocado em qualquer linha disponível da cache

Já no caso associativo por conjunto, cada bloco de memória é endereçado para um conjunto fixo, ocupando qualquer uma das k linhas disponíveis dentro do conjunto.

Considerando cache de 16 palavras, 1 palavra por bloco (16 linhas)

Cada palavra apresenta 4 bytes, portanto 2 bits são necessários (byte offset). Dentro de cada bloco, apenas uma palavra se encaixa, logo não precisamos de bits para blocos (word offset).

No mapeamento direto, como temos 16 linhas ($=2^4$), precisamos de 4 bits para localizar um bloco (índice).

No mapeamento associativo por conjunto ($k=4$), temos $16/4 = 4$ conjuntos $= 2^2$. Logo precisamos de 2 bits para localizar um bloco (índice)

Para cache com 32 palavras, 1 palavra por bloco (32 linhas).

- byte offset: 4 bytes por palavra $= 2^2$, logo 2 bits necessários
- word offset: 1 palavra por bloco, logo 0 bits necessários
- mapeamento direto: 32 linhas $= 2^5$. Logo 5 bits para índice
- associativo: sem índice (0 bits)
- associativo por conjunto: $32/4=8=2^3$. Logo 3 bits para índice

Para cache com 64 palavras, 1 palavra por bloco (64 linhas)

- byte offset: palavra de 4 bytes ($=2^2$). Logo 2 bits
- word offset: 1 palavra por bloco. Logo 0 bits
- direto: 64 linhas (2^6). Logo 6 bits de índice
- associativo: Sem índice (0 bits)
- associativo por conjunto: $64/4=16=2^4$. Logo 4 bits de índice

Para cache com 128 palavras, 2 palavras por bloco (64 linhas)

- byte offset: 2 bit
- word offset: 2 palavras por bloco ($=2^1$). Logo 1 bit
- direto: 64 linhas ($=2^6$). Logo 6 bits de índice
- associativo: sem índice. Logo 0 bits
- associativo por conjunto: $64/4=16=2^4$. Logo 4 bits de índice

Para cache com 128 palavras, 4 palavras por bloco (32 linhas)

- byte offset: 2 bits
- word offset: 4 palavras ($=2^2$). Logo 2 bits
- direto: 32 linhas ($=2^5$). Logo 5 bits de índice
- associativo: sem índice (0 bits)
- associativo por conjunto: $32/4=8=2^3$. Logo 3 bits de índice

8. Após fazer a divisão do endereço, considere a cache (a) definida no exercício anterior e calcule em que posição da cache será mapeada cada uma das seguintes palavras de memória (não se esqueça que a memória é endereçada a byte, então tem que converter cada endereço de palavra em um endereço de byte)

2,3,11,16,21,13,64,48,19,11,3,22,4,27,6,11

Consideramos aqui o caso de cache de 16 palavras, 1 palavra por bloco (16 linhas). Cada linha de cache é obtida como

$$cache_i = bloco_j \mod linhaCache_m$$

Sendo $bloco_j = endereçoByte / totalPalavrasPorBloco$, neste caso $bloco_j = númeroDaPalavra$, com $endereçoByte = palavra \times tamanhoPalavra$

Logo, temos $(w_i \times 4) / 4 \mod 16 = w_i \mod 16$, com mapeamento dado por

9. Realize os acessos abaixo na hierarquia de memória, exatamente na ordem em que eles aparecem. Os **endereços para acesso estão a Byte**. O computador trabalha com **palavras de 32 bits** (4 bytes). Suponha que o conteúdo da palavra de memória é o endereço do 1º byte daquela palavra. Há três níveis de cache (L1, L2, L3) e a memória RAM. A **cache L1 é mapeada diretamente e tem 4 linhas. Cada bloco da linha tem 2 palavras. A cache L2 tem 8 conjuntos, é associativa por conjunto com 2 linhas por conjunto e cada bloco da linha tem 2 palavras. A L3 tem 16 linhas e é totalmente associativa. Cada bloco da linha tem 4 palavras.** A memória **RAM tem 1024 bytes** (não é necessário endereçar mais do que essa quantidade de memória). A política de substituição nas caches L1, L2, L3 é FIFO. As políticas de atualização são *write-back* e *write-allocate* para as três caches (use 1 para modificado e 0 para não modificado). Os acessos abaixo podem ser Leitura ou Escrita. Quando o acesso for Escrita, o conteúdo a ser escrito na palavra é o endereço do 1ª byte multiplicado por 10.

- Represente graficamente as caches L1, L2, L3 e RAM
- Indique a divisão do endereço da memória principal para cada cache
- Faça a simulação e indique a situação final dos conteúdos na L1, L2 e RAM.
- Indique para cada cache: quantidade de acessos, acertos e faltas

palavra	endereço	posição cache
2	8	2
3	12	3
11	44	11
16	64	0
21	84	5
13	52	13
64	256	0
48	192	0
19	76	3
11	44	11
3	12	3
22	88	6
4	16	4
27	108	11
6	24	6
11	44	11

Table 1: Caption

São acessadas palavras que começam nos seguintes endereços, **já em bytes**

Acesso	Tipo	End Dec 1º byte	End Hexa 1º byte	End em bin
1	Escrita	72	48	0001001000
2	Escrita	64	40	0001000000
3	Escrita	136	88	0010001000
4	Leitura	36	24	0000100100

[SOLUÇÃO]

Como a RAM apresenta 1024 bytes de memória, precisamos de $(1024 = 2^{10})$ 10 bits. Cada palavra têm 4 bytes, portanto (2^2) temos 2 bits para byte offset.

Cache L1 apresenta mapeamento direto. Temos 2 palavras por bloco (2^1) e 1 bit para word offset, com 4 linhas no total (2^2) com 2 bits de índice.

Dos 10 bits de acesso à RAM, temos $10 - [2_{indice} + 1_{word} + 2_{byte}] = 5$ bits para tag.

Cache L2 é associativa por conjunto (k=2), com 8 conjuntos $(2^3, 3$ bits de índice), 2 linhas por conjunto $(2^1, com 1 bit word offset)$. Dos 10 bits de acesso à RAM, temos $10 - [3_{indice} + 1_{word} + 2_{byte}] = 4$ bits para tag.

Cache L3 é associativa, com 16 linhas, com 4 palavras por bloco (2^2 , com 2 bits word offset). Logo para os 10 bits da RAM, temos $10 - [2_{word} + 2_{byte}] = 6$ bits para tag.